

Experience with Training a Remotely Located Performance Test Team in a Quasi-Agile Global Environment

André B. Bondi

Johannes P. Ros

Siemens Corporate Research, Inc.

755 College Road East

Princeton, NJ 08540

andre.bondi@siemens.com, johannes.ros@siemens.com

Abstract

We describe our experience of training a remotely located team of developers and testers to prepare and execute performance tests. The team is located in India. The lead performance engineer and the test project manager are based in New Jersey. The team members had little or no prior experience of performance testing. We describe how we overcame cultural differences and a large time difference to develop a performance testing team that is now functioning well with far less supervision than was required at its inception. Cultural differences included contrasting views on adherence to strict laboratory procedures and assumptions about the prior knowledge, experience, and expectations of working habits of the India-based and New Jersey-based teams. We show how these differences and organizational challenges were overcome with intensive on-site training, the use of twice-daily scrum meetings, the careful designation of team leaders and role players at the remote testing site, and, eventually, the development intensive use of automated tools to execute performance tests and track the results.

1. Introduction

We describe our experience of training a remotely located team of developers and testers to prepare and execute performance tests. The team is located in India, while the lead performance engineer (the first author) and project manager (the second author) are based in New Jersey.

The team members had little or no prior experience of performance testing. For this reason, the performance test plan was designed and written in New Jersey, but executed in India. Apart from the predictable problems of dealing with cultural differences and a large time difference, this setup posed a wide variety of challenges, including the need

to explain measurement tools, performance testing practices, measurement procedures, rudimentary relationships between performance measures, and the need for documentation of all performance testing tools and experiments, all from afar. Initially, stringent deadlines and the uneven experience levels of the team members made it necessary to provide training on performance testing on the fly, at first remotely and then during a site visit by the lead performance engineer. We shall describe the training and working methods needed to deliver initial performance results and set the team on a path to the correct execution of a test plan, as well as some of the obstacles encountered. We shall also provide a narrative of how the performance effort progressed and how we responded to measurements and observations as they unfolded, together with a description of how the team functions nearly two years after its creation.

Among the obstacles encountered were the need to inculcate good performance testing practices, and the discovery that part of the system had concurrency bugs that needed to be patched before other tests could be executed. Even though the System Under Test (SUT) was not developed according to an agile process, we were able to use something like an agile process to move the performance tests forward. During a site visit by the performance engineer in the early stages of the performance testing project, twice-daily scrums facilitated communication between team members on progress and action items to be completed before the next scrum meeting, while enabling team members to volunteer information and help where needed to address blockages. The scrums also provided teaching opportunities in which the day's occurrences and observations were used as points of departure to explain basic performance analysis and testing concepts as well as the use of measurement tools.

As time went on, the team became more proficient at executing performance tests. With direction from

the project manager, a process and tools were developed to automate performance tests and track the results. This resulted in very short turnaround times for performance tests, allowing more time for root cause analysis on failing test cases than might otherwise have been available.

After the team hit stride, the frequency and length of regular team meetings were considerably reduced and replaced by individual meetings and calls with the project manager to deal with questions raised by individual team members.

The remainder of this paper is organized as follows. After mentioning related work on global software development and performance testing, we shall discuss the organizational challenges we faced in our project and performance aspects of agile methods. We shall then present a discussion of training issues related to performance testing, and then describe how our experience played out.

2. Related Work

In the course of this project, we applied techniques from a wide variety of disciplines. Performance measurement and modeling are mature fields about which much has been written for at least the past 30 years (see, e.g., the work on basic performance modeling described in [1] and the references cited there). A recently published book [2] gives comprehensive guidelines of performance testing methods and procedures for web-based systems, many of which would be applicable to more general systems as well. An example of an early description of the software lifecycle is given in [3]. Cultural aspects of global software development are discussed in [4]. Agile aspects of global software development are discussed in [5]. Scrum team management is described in [6] and elsewhere.

3. Organizational and System Challenges

3.1 Teaming Issues

The performance test team members had about ten members, with bachelors and/or master's degrees in computer science or electrical engineering. They were so focused on their individual responsibilities that lack of awareness of the issues their colleagues confronted impeded progress. To overcome this and to be able to rapidly deal with obstacles as they arose, the performance engineer organized twice-daily scrum meetings at which team members gave succinct reports on what they had done since the previous meeting, what obstacles had been encountered, and what they planned to do next. In conventional Agile processes, scrum teams are meant to be self-

organizing. This would not have been effective here, at least not until the team members gained the confidence to speak up and the experience to make judgment calls about how to overcome problems. Instead, we used a directed scrum format, in which the performance engineer acted as a moderator and asked groups of team members to work together to sort out issues as they emerged. Eventually, the team got to the point where they were able to volunteer solutions to each other's issues. The more senior members of the team told the performance engineer privately that they liked the scrum meeting system because they were more aware of their colleagues' activities than before, and because they felt better organized than they were prior to the performance engineer's arrival.

3.2 Working With Development and Other Teams

While the main purpose of our tests was to determine whether performance requirements could be satisfied, our performance tests revealed design shortcomings and concurrency and other bugs. Fixing these was outside the scope of the performance testing team. Nevertheless, the performance testers had to learn to support the developers and other stake holders to help them fix the problems their tests revealed. Support consisted of supplying documents such as dumps, traces, error logs, test specifications, and test reports, and descriptions of what occurred in greater detail than might be accommodated by a trouble ticket or modification request. The support activity is inherently interdisciplinary, because the causes of quantitative behaviour must be identified, and the behaviour might be explained in terms that are unfamiliar to the developers. The support task was facilitated by the introduction (after about a year) of a Wiki containing performance test results, analyses, and supporting materials, and by the use of a home-grown system for executing and tracking performance tests and storing them on a server that was remotely accessible. This in turn enabled the quick turnaround of test results by the performance test team while developers and others in different time zones were off duty.

3.3 Identifying Leaders, Interpersonal Dynamics, Skill Sets

For the performance testing effort to be successful, it was necessary to determine who works well with whom, to distinguish between expressions of enthusiasm on the one hand and discipline and commitment on the other, and to figure out how best to harness individuals' skills, including leadership

skills, to get the job done. It was very difficult to do this via the conference calls and phone conversations with individuals held prior to the performance engineer's site visit, especially when one individual was designated as the liaison between the team and the performance engineer. Individuals' qualities were much more apparent to the performance engineer after he had been working on site for just a few days than they had been before.

3.4 Cultural Challenges

The performance engineer's experience with cultural challenges, particularly those relating to initiative regarding problem resolution, literal adherence to requirements, inquisitiveness about domain knowledge, and the like was consistent with what has been reported elsewhere [7]. Recognizing signs of comprehension and incomprehension was a bit difficult at first. At the risk of irritation, continuous follow-up and questioning were needed to ensure that work was being carried out as desired. This might have been all but impossible had the performance engineer not been on site.

3.5 Management and Communications Challenges

To ensure that the team did not drift from its newly acquired discipline and focus, and to keep it on track, the project manager found it necessary to have frequent telephone conversations with team members, even once the team had hit stride after about a year. These conversations consume about 20 hours a week, and usually take place between 6:30 a.m. and 10:30 a.m. New Jersey time. Not all team members took part in all conversations at the same time, as this would have been disruptive to their work. Moreover, it is felt that individual conversations reinforce a sense of ownership on the part of individual team members. Group meetings were only held to deal with administrative matters, planning, and at milestones.

3.6 System Challenges

The SUT is a service platform with about two million lines of code. Because the performance test team was the first to use the platform from end to end and the first to integrate it as a platform user would, it was the first to run into obstacles that developers and unit testers had not anticipated, such as concurrency bugs. The team had to come up with workarounds to enable the tests to continue. Before this could be done, test cases had to be devised on the fly to verify that the cause of the problem was the one suspected. A side

benefit of the performance team's crafting the first end-to-end use is that problems were caught and fixed internally before the system was released to a customer.

4. Performance Testing and Agility

Each release of the SUT is developed according to a V model process [8] [3], with a progression from concept to requirements to architecture, followed by a progression to design, implementation, functional testing, and, ultimately, end-to-end performance testing. The SUT is a complex service-oriented XP-based platform that provides services for various applications, implemented in a mix of C and C++. Because performance tests cannot be done until functional tests have been completed, they are always the last stage in any development life cycle before a release can be declared complete. The performance tests are usually also the first time that the concurrent aspects of system functionality are exercised, and hence the first opportunity for bugs in concurrent programming and scheduling policies to appear.

Although the development process for this project was not agile, agile practices could be adapted in devising, implementing, and executing performance tests to some extent. For instance,

- Twice-daily scrum meetings meant that problems could be resolved quickly, and that incorrect courses of action could be altered. They also meant that team members were motivated to make rapid progress, because they had to account for their time twice a day. The need to describe what team members had done gave them an incentive to document their work. In addition, the scrum format enabled the team members to personally divorce themselves from the issues they encountered, because each member could see that others were encountering problems, too.
- The development of the test environment, supporting measurement instrumentation, and test scripts could commence even in the absence of a complete test plan or complete performance requirements. This was done once functional specifications, interfaces, and the deployment architecture had been largely specified. This ensured that the performance test environment would be nearly ready once development and functional testing were nearly complete.

- We incrementally tested several components of the performance test environment before the performance test cases were executed. This was done to ensure
 1. that correct data was being offered to the application at the right rates,
 2. that system resource usage measurements and statistics about response times were being correctly collected, and
 3. that instrumentation and load generators were correctly calibrated.

5. Training Challenges and Content

5.1 Initial Conditions

The pressure of deadlines meant that there was no time to provide classroom training to the team on the rudiments of performance engineering and performance testing practice. This training had to be provided on the fly through lunch time conversations, as adjuncts to scrum meetings, and through an insistence on adherence to procedures laid down by the lead performance engineer. At the same time, it seemed necessary to balance the time taken to motivate the team and its work by telling them about performance engineering and testing principles against telling them the minimum they needed to know make sufficient progress with the performance test plan, especially since the lead performance engineer planned to spend only a limited amount of time in India.

5.2 Test Practice and Procedure

We could not make assumptions about team members' knowledge about laboratory practice.

The following is a list of points that could not be taken for granted during interactions with the team. The performance engineer underscored them by pointing out that one would have (or should have) followed the same experimental procedures in high school courses in physics and chemistry.

1. *The need to keep a laboratory notebook detailing experimental setups and results.* This was essential to ensuring that observations were not lost, and that correct procedures were followed. Later in the project, handwritten notes were transcribed to a Wiki after each experiment.
2. *How to take detailed measurements.* The team was supplied with a detailed list of

resource usage measurements to capture, but had to be trained to use Windows *Perfmon* to do so.

3. *The need for "Clean Test Tubes".* We had to emphasise that the SUT should not have any unrelated activity going on during the measurement period. The same holds for hosts supporting load generators.
4. *Load generators should be run on separate hosts to keep measurements of the SUT clean.* Some might prefer to run the load client on the same host as the SUT. This is not recommended because of the risk that memory and I/O interference will increase response times, even though it might be possible to separate its CPU usage from that of the application.
5. *A check list should be generated and run through for each and every test case, prior to, during, and after execution.* The list should describe the configuration parameters, the name of the use case, the sets of measurements, the start and stop times of the test run, the names of the log and configuration files used, and notes on issues and error messages encountered during the test.

Protests about lack of time for these procedures were firmly resisted, on the grounds that adherence would save time later by minimizing repeated work and reducing the need to forage for documentation of test conditions. Later in the project, automation relieved much of the burden of these procedures.

5.3 Providing Context for Performance Tests

To maintain focus and alertness, the team needed to have a context for the performance tests.

- The test team had little knowledge of how the system was eventually going to be used. This was dealt with by describing some concrete examples and relating them to the test case scenarios.
- The algebraic relationships between offered load, utilizations, queue lengths, and average response times are key to understanding performance test data and using it to diagnose problems [1]. The tests must be structured to facilitate verification that the relationships do indeed hold. This was dealt with by giving a short lecture on the relationship between utilization, response

time, and offered transaction rate at the end of a pre-lunch scrum meeting, with slides.

- The team needed to understand that average performance measures should be quite constant under constant load once the system has ramped up.
- The team needed to understand that the CPU utilization is proportional to the offered load, provided that it is zero when the system is empty and idle before the test.
- The team needed to understand that (a) that there was no point in increasing the offered test load to a point at which the CPU or any other component would be 100% busy, and that a utilization of 100% was potentially bad news, especially when no payload was going through the system.
- The essential role of the performance tests in improving the quality of the project became more apparent to the performance testing team over time. The team acquired more confidence and motivation about detecting performance-affecting bugs and reporting them to the development team. The development team eventually became more aware of performance testing team's ability to help them trouble-shoot problems, and was more attentive to their reports.

5.4 Explaining How to Identify Abnormalities in Software Performance Measurements

Inexperienced testers must learn how to identify abnormalities occurring in system measurements and/or the instrumentation used to collect and gather them. Recognising abnormalities is a cross between a science and a craft. The India team encountered the following aspects of this:

- Poor performance and/or excessive resource consumption are often the first evidence of a defect in the system. It was necessary to explain that to get the most out of a performance test, one should not only check whether performance requirements have been met, but also check for such signs of abnormality as crashes and widely varying response times in the presence of low processor utilization. Knowing what to look for takes experience. The challenge was sharing the experience without overloading

overworked and/or disorganized performance test engineers.

- Measurement tools can be faulty, and/or ambiguously and incorrectly documented, or both. Simple, controlled experiments may be needed to determine the exact meaning of a measurement, and what it tells us about how the operating system and the application behave. We worked closely with the performance testers to identify these problems and trained them to recognize them and sort them out.

6. Evolving Practices

6.1 Automating Checklist Verification, Configuration, and Logging

Because the checklists can be lengthy and complicated, executing them was eventually automated as much as possible. This is faster than manual checking, and less error prone. Automation of test bed configuration, execution, and logging reduced the difficulty of enforcing strict record keeping, especially once the team obtained results and wanted to try out many different parameter settings to see what would happen. Automation also enabled the binding and auditing of parameter and configuration files to make sure they matched.

6.2 Recording Lab Notes in a Wiki

To facilitate the sharing of information between team members, developers, and other stakeholders, the team eventually established the practice of transcribing their hand written lab notes onto a Wiki. The Wiki could be used to store related materials, such as error logs and test reports. In addition, the Wiki enabled the efficient sharing of information among stakeholders in different time zones, facilitated root cause analysis, and reduced the volume of e-mails and the time to turn around requests for further information.

7. How Testing and Training Proceeded

7.1 Early Site Visit

Before load could be directed at the SUT, the testers had to develop load generators from scratch, and learn how to use basic performance measurement tools such as NT Perfmon. Clock synchronization and timing issues had to be resolved, along with procedures for moving and storing performance and

associated error logs. Basic lab procedures had to be established. Because equipment could not be locked in a separate room, an attempt was made to safeguard it from tampering by taping “Test Equipment – Do Not Touch!” notices to the monitors. The test engineers had to craft configuration files for the system, and make sure that everything worked.

The team took longer to progress than expected. Therefore, the performance engineer had to extend his stay in India by a week, and insist on a Saturday work session to maximize lab time while he was there. That entailed logistical problems such as providing car service on days when company-sponsored commuter buses were not running and managing to work with the air conditioning turned off and windows open in a city with heavy air pollution.

Happily, the Saturday exercise was very fruitful. The team was delighted when applying load to the system resulted in a visible increase in CPU utilization, and when turning the load off resulted in a drop. Unfortunately, starting and finishing average CPU utilizations were not zero, but 50%. This provided a teachable moment. Under the guidance of the performance engineer, the team learned the value of collecting the utilizations of individual processors when one was shown to be 100% and the other 0% in the absence of applied load. Further investigation showed that one process was the culprit. One of the test engineers recognized that the offending process was repeatedly polling an empty message interface, and that neither the requirements nor the architecture had allowed for any other kind of inter-process communication.

At this point, the team lost interest in documenting its work and wanted to try out all sorts of things. This sort of play time was conducive to their getting a feel for how performance measures and instrumentation work. After struggling to record what the team was doing and the results, the performance engineer called a halt and gathered the team members in a conference room. We recorded observations, possible causes, and possible avenues for investigation in separate columns on a whiteboard, transcribed them into a spreadsheet, and developed an action plan for the following business day. The plan included sending e-mails to the system development team (located in Germany) and systematically conducting short tests with different parameters under tightly controlled conditions, with careful documentation. The upshot was the identification of some serious bugs in both the software and the underlying requirements. Change requests were written against them on the following Monday afternoon. The team could not proceed with the execution of test cases until they had identified a workaround to the most obstructive problem and

implemented it. This experience provided a valuable lesson that obstacles encountered in the execution of a test plan can provide valuable insights into the way the system behaves. It also showed the team that performance testing is extremely valuable for provoking concurrency and scheduling problems which could not have shown up in unit testing or if everything had gone smoothly.

7.2 Aftermath of the Early Site Visit

The performance engineer’s involvement with the India team was taken over by the project manager after he returned home. The team eventually completed the test plan successfully and has since embarked on the implementation and execution of performance test plans for other releases, some with more elaborate deployment scenarios. The more astute members of the team are now exercising stronger leadership roles in the performance testing process than they did previously, and progress continues.

7.3 Sequel

In addition to frequent conference calls, further intermittent on-site involvement by New Jersey-based performance engineers and architects has been needed and continues to occur to ensure that momentum is sustained. More than a year after the lead performance engineer’s site visit, the performance testing team is functioning with a great deal more autonomy and effectiveness than it did at its inception. New sets of performance test cases on subsequent releases have been executed with far less intervention than before. The intense use of automated testing has facilitated quick turnaround of tests of modifications made by developers. The automated archiving and remote accessibility of the results have enabled developers, architects, and the project manager to view and track the test results from their offices in Princeton and Germany, or wherever they happen to be.

8. Omissions from Training

While the India team worked much more cohesively later in the project than at the beginning, there were a number of aspects of performance testing and engineering that it was not trained to do, though some might be encouraged to try.

- There was no training in the specification of test cases, because the team initially lacked the domain knowledge and knowledge of performance testing to do so fruitfully. In addition, successful design of performance tests requires some understanding of experimental design, such as factor

identification and suitable choices of performance metrics. The test cases were drafted by an architect with assistance from the performance engineer.

- During the initial site visit by the performance engineer, the team was not trained in the use of pilot tests and simple performance models to plan experiments. The pilot test can be used to establish whether a particular transaction rate or load scenario exercises the system enough to generate visible resource utilization, but not so much as to saturate it. If saturation occurs, the system should be tested with a much lower load. A performance tester should be trained to recognize feasible load ranges so as to avoid wasting lab time on multiple loads in excess of one that causes saturation anywhere in the system.
- The performance team was not taught how to identify meaningful performance metrics as an aid to choosing test scenarios. Transferring this knowledge to a team requires formal training, which time constraints did not allow. In the performance engineer's experience, many developers will assess the throughput of a system by subjecting it to as fast an uncontrolled load as possible and measuring the highest completion rate observed before the system crashes or before a large increase in response times is observed. This kind of experiment fails to tell us whether resource utilizations are linear in the offered load. It also fails to reveal the knee of the response time curve, which is crucial to identifying the maximum sustainable load and to determining whether there is a software bottleneck in the SUT.

9. Lessons Learned

9.1 The Benefit of Domain Knowledge

We found that the focus of the team and its alertness to the importance of oddities in the test data were improved once they had knowledge of the domain of application of the platform. Until then, they had insufficient context and awareness of the use of the system to be aware of the importance of their test results.

9.2 Play Time

The team acquired a better feel for what was happening in the system by playing with it in spontaneous manner for an hour or so. They could see the performance impact of sending in requests at different rates, and try out different *perfmom* counters to see which ones might yield insights and which ones not. The performance engineer found that play time with the system improved the team's morale and members' willingness to take the initiative to try out different kinds of tests. It also provided teachable moments. Improving morale was particularly important when the team was pressed to work outside normal business hours.

9.3 Automated Completion of Check Lists

The team resisted repeated checking of configuration settings that did not change from one test run to the next. Eventually, the team was encouraged to use scripts to automate the checking and recording of parameter settings where possible. This reduced test preparation time while maintaining test quality.

9.4 Automation on the Path to Excellence

The performance test team was not skilled in the art of performance testing when it was initially convened. In particular, it lacked laboratory discipline and knowledge of performance engineering *per se*. Once the team had a feel for how performance testing was to be done, automation of many of the practices we wanted to put in place improved both the morale and the discipline of the team. Automation also enabled the team to spend more time on examining the test results and explaining them to developers, and to quickly respond to developers' and other stake holders' requests for tests with different configuration and load parameters.

9.5 Wiki to Archive Lab Notes, Test Results, and Reports

Using the Wiki greatly facilitated information sharing within the team, between the teams, and across multiple time zones. It was used by technical staff to aid in problem resolution, and by managers to track how the project was going. It also improved management's understanding of difficulties encountered in the project while reducing e-mail traffic between stake holders.

9.6 Interdisciplinary Work Between Teams

The performance test team frequently had to interpret its results to developers and other stakeholders, especially when the results indicated the presence of a bug or design flaw. This is an acquired skill. As mentioned above, the Wiki facilitated the information exchange necessary to do this.

10. Side Observations

Our experience with training a performance testing team raises questions about the type of educational background that best prepares one to be a member of a performance team. This should be born in mind when building one, especially if the team must be ramped up quickly. When the performance engineer first became involved with performance evaluation and measurement, his mentors were physicists and economists. Formal training in programming was not as widely available then as it is now. When he first taught computer performance evaluation and simulation at UCSB, he found that the students who related modeling predictions to system behaviour the best were those who had degrees in biology, chemistry, or physics. They applied the thinking and habits of their disciplines to a new area. By contrast, the computer science majors did not seem to be as used to quantitative reasoning about physical phenomena, and did not take as much initiative in their investigations as the natural science majors did. They also had (and still tend to have) very little experience of lab work involving systematic measurement and discovery. It would be interesting to see whether a systematic study of the backgrounds of performance engineering teams would yield similar observations.

11. Conclusions

We have described our experience with training a remotely located performance testing team to implement and conduct performance tests. We found that the team needed a good deal of education and training in aspects of performance testing principles that many would consider rudimentary, and that enthusiasm and commitment are not guarantors of adherence to correct testing procedures. Because of the lack of time, choices were made about what knowledge of testing principles had to be conveyed on the fly to give momentum to the performance testing exercise. Scrum meetings were a very effective tool for fostering communication between members and resolving problems quickly. They also provided a useful opportunity to provide training on

performance testing basics as needed, and to match tasks to skills. Despite the ability to look at computer screens remotely, the lead performance engineer's presence on site was necessary to give the team sufficient training and momentum to complete the performance test plan in an organized manner. Further on-site involvement by the project manager has provided additional training opportunities. Automated performance testing and configuration and the use of a Wiki to share performance test results and lab notes have facilitated information sharing. They also reinforced the notions of and added to the benefits of careful record keeping. In addition, automation enabled rapid turnaround time and enabled the team to better support the development team in identifying the causes of performance problems as they were identified. The team is now much more productive than it was at its inception.

12. References

- [1] Denning, P.J., and J. P. Buzen, The operational analysis of queueing network models, *ACM Computing Surveys* **10(3)**, pp. 225-261, 1978.
- [2] Meier, J. D., S. Barber, C. Farre, P. Bansode, and D. Rea: *Performance Testing Guidance for Web Applications*, Microsoft, 2008.
- [3] Boehm, B. A spiral model of software development and enhancement. *IEEE Computer* 21(5), 61-72, 1988.
- [4] Holmstrom, H., E. O. Conchuir, P.K. Agerfalk, and B. Fitzgerald: Global Software Development Challenges: A Cast Study of Temporal, Geographical, and Socio-Cultural Distance. *Proc. 2006 IEEE Conference on Global Software Engineering*, pp 3-11, 2006.
- [5] Paasivaara, M. and C. Lassenius, Could Global Software Development Benefit from Agile Methods? *Proc. 2006 IEEE Conference on Global Software Engineering*, pp 109-113, 2006.
- [6] Schwaber, K., and Beedle, M. *Agile Software Development with Scrum*, Series in Agile Software Development, 2001.
- [7] Abraham, L. R., Cultural Differences in Software Engineering, *Proc. ISEC'0*, pp. 95-99, 2009
- [8] Die Beauftragte der Bundesregierung fuer Informationstechnik, *Da V-Modell XT*, <http://www.v-modell-xt.de/>, Version 3 February 2009.